

MERN STACK ARCHITECTURE & INTERVIEW GUIDE

Eventora — Technical Documentation

A high-performance event discovery and booking platform engineered with role-based access control, two-factor OTP verification, resilient transactional mail pipelines, and a dynamic glassmorphic UI.

ARCHITECTURE	AUTH MODEL	STYLING	TARGET ROLE
Full-Stack MERN	JWT + 2FA OTP + RBAC	Tailwind CSS v4 + Dark/Light	Full-Stack / MERN Engineer

01. Tech Stack & Component Architecture

Frontend Core

- **React 19 & Vite 8:** Instant HMR and high-efficiency tree-shaking.
- **Tailwind CSS v4:** Dark mode via `@custom-variant`.
- **React Router v7:** Declarative client-side routing.
- **React Icons:** Lightweight icon library.

Backend Services

- **Node.js & Express:** RESTful API server.
- **Bcrypt.js:** One-way password hashing (10 rounds).
- **JSON Web Tokens:** Stateless bearer authentication.
- **CORS Whitelist:** Domain-restricted credential policies.

Database & Mail

- **MongoDB Atlas:** Managed document datastore.
- **Mongoose ODM:** Schema validation & population.
- **Smart Dual-Engine Mailer:** Nodemailer + HTTPS API failover (Brevo & Resend).

Project Structure Overview

DIRECTORY	KEY FILES	RESPONSIBILITY
client/src/context/	AuthContext.jsx , ThemeContext.jsx , ToastContext.jsx	Modular global state handling user sessions, dark/light theme persistence, and notifications.
client/src/pages/	Home , EventDetail , UserDashboard , AdminDashboard , Login	Responsive views: debounced search, event passes, admin counters, and verification modals.
server/controllers/	authController.js , eventController.js , bookingController.js	Core business logic: seat calculations, OTP generation, password hashing, and booking approvals.
server/middleware/	auth.js (protect , admin)	Route guards: Bearer token validation and Role-Based Access Control (RBAC).
server/utils/	email.js , generateOTP.js	Dual-channel transactional email engine with HTTPS port 443 failover.

02. Core Architectural Lifecycles

A

Authentication & 2FA Verification Flow

- 1. Registration:** Client sends `name, email, password`. Server hashes password with `bcrypt`, inserts user with `isVerified: false`, creates a 6-digit OTP in the `OTP` collection (5-minute TTL), and dispatches a verification email.
- 2. OTP Verification:** Client submits OTP to `/api/auth/verify-otp`. Server validates the document, flags `isVerified: true`, purges the OTP record, and responds with a signed JWT.
- 3. Login Gate:** If an unverified user logs in, the API returns a `needsVerification: true` flag, automatically triggers a fresh OTP dispatch, and shifts the UI into OTP mode without revealing internal errors.

B

Two-Factor Booking & Ticket Approval Flow

- 1. Intent & OTP Dispatch:** Logged-in user triggers booking. Server deletes stale OTPs, creates a new `event_booking` OTP, and emails it.
- 2. Reservation Submission:** User submits OTP + `eventId`. Server checks `availableSeats > 0` and ensures no active booking exists for this user-event pair. Creates booking with `status: "pending"`.
- 3. Admin Confirmation:** Admin reviews request via `PUT /api/bookings/:id/confirm`, flags `status: "confirmed"`, sets `paymentStatus: "paid"`, decrements `availableSeats`, and asynchronously triggers a confirmation email.
- 4. Ticket Generation:** User Dashboard renders a visual Event Pass ticket with side tear notches, ticket ID, holder details, and printable QR code via `window.print()`.

03. Key Engineering Challenges Solved (Interview Highlights)

1. Cloud Host SMTP Outbound Port Blocking Bypass (Port 443 Fallback)

Challenge: Cloud providers like Render block standard outbound SMTP ports (25 , 465 , 587) on free instances to prevent email abuse, causing standard Nodemailer sockets to hang and timeout.

Engineered Solution: Implemented a smart dual-engine mailer in `server/utils/email.js`. The mailer inspects environment variables:

- If `EMAIL_PASS` starts with `re_` (Resend) or `xkeysib-` / `xsmtpsib-` (Brevo), it circumvents raw sockets and calls their **HTTPS REST APIs over Port 443** via native `fetch`.
- If using standard SMTP credentials (e.g., local development with Gmail), it gracefully falls back to standard Nodemailer transport.

2. CSS Containing Block Trap Solved via React Portals

Challenge: The user Profile popup inside the navbar was clipped and constrained within the navbar's boundary rather than covering the whole viewport.

Root Cause: Under W3C CSS specifications, applying `backdrop-filter: blur(...)` on a parent element creates a new containing block for all its `fixed` descendants, breaking traditional modal positioning.

Engineered Solution: Used `ReactDOM.createPortal(modalJSX, document.body)` in `ProfileModal.jsx`. This mounts the modal outside the navbar's DOM tree directly to `<body>`, allowing full-screen backdrops and reliable outside-click dismissal.

3. DNS SRV Lookup Timeouts for MongoDB Atlas

Challenge: Local ISP and cloud DNS servers occasionally fail when resolving `mongodb+srv://` connection strings, resulting in `querySrv ENOTFOUND` errors.

Engineered Solution: Explicitly registered fallback DNS resolvers in `server/index.js` prior to connection:

```
const dns = require("dns");
dns.setServers(["1.1.1.1", "1.0.0.1"]); // Cloudflare Public DNS
```

4. Concurrency & Race Conditions in Seat Reservations

Interview Discussion Point: How does the system handle concurrent bookings when only 1 seat remains?

Implementation & Scalable Evolution:

- **Current:** Two-phase validation — verified at OTP submission and again during Admin approval.
- **High-Scale Solution:** Use MongoDB atomic operator with conditional query:

```
await Event.findOneAndUpdate(
  { _id: eventId, availableSeats: { $gt: 0 } },
  { $inc: { availableSeats: -1 } }
);
```

This guarantees database-level atomicity without needing distributed locks.

04. Database Schema Architecture

COLLECTION	FIELD	TYPE	CONSTRAINTS & NOTES
User	name	String	Required, trimmed
	email	String	Required, unique index, lowercase

COLLECTION	FIELD	TYPE	CONSTRAINTS & NOTES
	password	String	Hashed with bcrypt (10 rounds), excluded on query select
	role	String	Enum: ["user", "admin"], default: "user"
	isVerified	Boolean	Default: false . Toggled to true post-OTP verification
Event	title , desc , loc	String	Required descriptive metadata
	category	String	Enum filterable: Tech, Music, Business, Art, Food, etc.
	date	Date	Required event timestamp
	totalSeats , availSeats	Number	Available seats decremented upon booking confirmation
	ticketPrice	Number	0 signifies Free Entry pass
Booking	userId , eventId	ObjectId	Refs to User and Event models
	status	String	Enum: ["pending", "confirmed", "cancelled"]
	paymentStatus	String	Enum: ["nonpaid", "paid"]
	timestamps	Date	Auto-generated createdAt and updatedAt
OTP	email , otp	String	6-digit numeric string
	action	String	Enum: ["register", "event_booking"]
	expiresAt	Date	TTL expiration timestamp (5 minutes)

05. REST API Reference Matrix

METHOD	ENDPOINT	ACCESS	PAYLOAD / PARAMS	DESCRIPTION
POST	/api/auth/register	Public	{ name, email, password }	Registers user & sends 2FA registration OTP
POST	/api/auth/login	Public	{ email, password }	Authenticates user. Rejects if unverified
POST	/api/auth/verify-otp	Public	{ email, otp }	Validates registration OTP & returns signed JWT
PUT	/api/auth/profile	User (JWT)	{ name }	Updates authenticated user's display name
GET	/api/events	Public	Query: search , category	Returns list of events with debounced filter support
GET	/api/events/:id	Public	Params: id	Returns detailed event metadata & remaining seats
POST	/api/events	Admin (RBAC)	{ title, date, seats, price... }	Creates new event listing
DELETE	/api/events/:id	Admin (RBAC)	Params: id	Deletes event record from database
POST	/api/bookings/send-otp	User (JWT)	Headers: Authorization	Generates and emails booking confirmation OTP
POST	/api/bookings	User (JWT)	{ eventId, otp }	Verifies OTP & creates pending booking
GET	/api/bookings	User / Admin	Headers: Authorization	Returns user's bookings (or all if admin)
PUT	/api/bookings/:id/confirm	Admin (RBAC)	{ paymentStatus }	Confirms booking, decrements seat, emails user
DELETE	/api/bookings/:id	User / Admin	Params: id	Cancels booking request

06. Top Interview Questions & Comprehensive Answers

Q1: How do you design and secure the authentication architecture in Eventora?

"Authentication is stateless and uses a two-phase model:

- Password Encryption:** Passwords are never stored as plaintext; they are salted and hashed using `bcryptjs` (10 rounds).
- Mandatory 2FA Email OTP:** Accounts remain in an unverified state until the user validates a 6-digit OTP stored in MongoDB with a 5-minute TTL.
- JWT Token Management:** Once verified, an encrypted JWT bearer token is signed containing the user ID and role. Sensitive fields like passwords are systematically projected out (`.select('-password')`) in database queries.
- Role-Based Access Control (RBAC):** Custom Express middleware (`protect` and `admin`) decodes tokens and guards sensitive endpoints like event creation and booking approvals."

Q2: How did you optimize frontend performance and handle state management?

"I avoided bloated third-party state managers by utilizing **modular React Contexts**:

- **AuthContext** handles session persistence and user login/logout states.
- **ThemeContext** manages light/dark mode persistence via **localStorage** and root class toggling.
- **ToastContext** renders animated alerts without blocking the render tree.

To optimize performance, search queries on the Home page are **debounced by 400ms**, eliminating redundant network calls on rapid keystrokes. Vite handles chunk bundling for minimal bundle footprint."

Q3: What security measures did you implement against injection and unauthorized access?

"1. **CORS Origin Whitelisting**: Dynamic CORS validation ensures only trusted origins (e.g. configured frontend domain and local development ports) can interact with credentials enabled.

2. **Mongoose Schema Casting**: Built-in schema validation protects against NoSQL injection.

3. **Sanitization**: Trimmed environment variables to prevent newline injection bugs in API keys.

4. **Stateless Access**: API tokens cannot be forged without the server's private **JWT_SECRET**."

Q4: If you were taking Eventora to 100,000 concurrent users, what would you re-architect?

"1. **Caching Layer**: Move OTP storage and frequently queried events to **Redis** with automatic memory key expiration, offloading MongoDB write traffic.

2. **Asynchronous Event Queue**: Offload email delivery to a background message queue like **BullMQ / RabbitMQ** with retry policies so HTTP request-response cycles are never blocked by external mail APIs.

3. **Atomic Seat Decrements**: Use MongoDB atomic operations (**\$inc** conditional on **\$gt: 0**) or database transactions to prevent race conditions during flash ticket releases.

4. **CDN & S3 Storage**: Offload uploaded event banner images to an S3 bucket with Cloudflare CDN caching."